

COURSE 5: PYTHON PROJECT FOR DATA SCIENCE

MODULE 1: CROWDSOURCING SHORT SQUEEZE DASHBOARD

1.1 COURSE INTRODUCTION

PURPOSE OF THE MINI-COURSE

- This project-based course is a **continuation of**:
 - *Python for Data Science, AI and Development*
- You apply previously learned Python skills in a **realistic data science scenario**

YOUR ROLE

- You act as a **Data Scientist / Data Analyst**
- You will:
 - Extract data
 - Perform web scraping
 - Analyze and visualize data
 - Build a dashboard
 - Submit a **Python notebook** for **AI evaluation**

PREREQUISITES

- Prior knowledge of:
 - Python basics
 - Pandas
 - Web scraping
 - DataFrames
-

1.2 WEB SCRAPING – A KEY TOOL IN DATA SCIENCE

WHAT IS WEB SCRAPING?

- Also known as:
 - Web harvesting
 - Web data extraction
- Process of extracting **unstructured data** from websites
- Converts data into a **structured format** (tables, DataFrames)

IMPORTANCE OF WEB SCRAPING

APPLICATIONS

-  **Data Collection**
 - Gathering datasets from the internet
-  **Real-time Applications**
 - Weather updates
 - Price comparison
-  **Machine Learning**
 - Collecting training data
-  **Market & Trend Analysis**

1.3 WEB SCRAPING WITH PYTHON

KEY PYTHON LIBRARIES

Library	Purpose
requests	Download web pages
BeautifulSoup	Parse HTML/XML
Scrapy	Large-scale crawling
Selenium	Scraping dynamic (JS-based) sites

Python provides several libraries for web scraping. Here are some of them:

- **BeautifulSoup:** BeautifulSoup is a Python library used for web scraping purposes to pull the data out of HTML and XML files. It creates a parse tree from page source code that can be used to extract data in a hierarchical and more readable manner.

```
from bs4 import BeautifulSoup
import requests
URL = http://www.example.com
page = requests.get(URL)
soup = BeautifulSoup(page.content, "html.parser")
```

- **Scrapy:** Scrapy is an open-source and collaborative web crawling framework for Python. It is used to extract the data from the website.

```
import scrapy
class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = ['http://quotes.toscrape.com/tag/humor/',]
    def parse(self, response):
        for quote in response.css('div.quote'):
            yield {'quote': quote.css('span.text::text').get()}
```

- **Selenium:** Selenium is a tool used for controlling web browsers through programs and automating browser tasks.

```
from selenium import webdriver
driver = webdriver.Firefox()
driver.get("http://www.example.com")
```

READING: WEB SCRAPING - A KEY TOOL IN DATA SCIENCE

1.4 HTML FOR WEB SCRAPING

WHAT IS HTML?

- **HyperText Markup Language**
- Defines the structure of web pages
- Data is enclosed inside **tags**

BASIC HTML STRUCTURE

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body></body>
```

<code></html></code> Element	Description
<code><!DOCTYPE html></code>	Declares HTML document
<code><html></code>	Root element
<code><head></code>	Metadata
<code><body></code>	Visible content (scraping target)

COMMON TAGS USED IN SCRAPING

Tag	Meaning
<code><h3></code>	Heading (player names)
<code><p></code>	Paragraph (salary)
<code></code>	Bold text
<code><a></code>	Hyperlink
<code><table></code>	Table
<code><tr></code>	Table row
<code><td></code>	Table cell

COMPOSITION OF AN HTML TAG

Anchor Tag Example

```
<a href="https://www.ibm.com">IBM</a>
```

Components

- **Tag name** → a
- **Opening tag** → <a>
- **Closing tag** →
- **Attribute** → href
- **Attribute value** → URL
- **Content** → IBM

Analogy

- Tag name → Python class
- Tag instance → Python object

1.5 HTML TREE (DOM STRUCTURE)

HTML documents follow a **tree hierarchy**.

KEY RELATIONSHIPS

- **Parent**
- **Child**
- **Sibling**
- **Descendant**

EXAMPLES

- <html> → parent of <head> and <body>
- <head> & <body> → siblings
- <title> → child of <head>
- → child of <h3>

This structure enables **BeautifulSoup navigation**.

1.6 WEB SCRAPING WITH BeautifulSoup

BeautifulSoup Objects

- Represent HTML as **tree-like objects**
- Each HTML tag becomes a **Tag object**

Navigation Methods

Method	Purpose
.parent	Move upward
.child	Move downward
.next_sibling	Move sideways
.attrs	Access attributes
.string	Extract text

Handling Multiple Tags

- If multiple tags exist:
 - First matching tag is selected by default

1.7 find_all() Method

Purpose

- Finds **all matching elements** based on filters:
 - Tag name
 - Attributes
 - Text
 - Combinations

Example

```
rows = table.find_all("tr")
```

- Returns a **list-like iterable**

- Each element is a **Tag object**

EXTRACTING TABLE DATA

```
for row in rows:
```

```
    cells = row.find_all("td")
```

```
    for cell in cells:
```

```
        print(cell.text)
```

1.8 SCRAPING LIVE WEB PAGES

1. Import libraries
2. Use requests.get(URL)
3. Extract page text
4. Create BeautifulSoup object
5. Navigate and extract data

```
import requests

from bs4 import BeautifulSoup

# Fetch the webpage
page = requests.get("http://EnterWebsiteURL").text

# Create BeautifulSoup object
soup = BeautifulSoup(page, "html.parser")

# Pull all <a> tags
artists = soup.find_all('a')

# Loop through each <a> tag
for artist in artists:

    name = artist.get_text(strip=True)

    fullLink = artist.get('href')

    print(name)

    print(fullLink)
```

WEB SCRAPING TABLES USING PANDAS

read_html() Function

- Extracts **all tables** from a webpage

```
import pandas as pd

URL = "https://en.wikipedia.org/wiki/List_of_largest_banks"

tables = pd.read_html(URL)

df = tables[0]
```

ADVANTAGES

- Very fast
- Automatically creates DataFrames

LIMITATIONS

- Scrapes **hidden tables**
- Retains hyperlinks & extra symbols
- Requires **data cleaning**
- Works **only for tables**

📌 Important

👉 BeautifulSoup is more flexible and remains the default scraping method.

ETHICAL WEB SCRAPING

- Follow **robots.txt**
- Respect website terms of use
- Avoid excessive requests

[READING: WEB SCRAPING TABLES USING PANDAS](#)

[INTRO TO WEB SCRAPING USING BEAUTIFULSOUP](#)

1.9 Practice Project: Extracting Stock Data Using yfinance

1. Project Overview

In this project, you act as a **Data Scientist** and work with stock market data.

You will learn how to:

- Extract stock price data using **yfinance**
- Extract revenue/profit data using **web scraping**
- Clean and analyze stock data
- Compare stock price vs company revenue
- Build a dashboard

2. What is a Stock Share?

A stock share represents **ownership in a company**.

Definition:

A stock (equity) gives you ownership of a small part of a corporation.

If you own shares, you are entitled to:

- Part of company assets
- Part of profits

Units of stock:

- Called **Shares**

3. How Investors Earn or Lose Money

Profit:

If stock price increases after buying.

Loss:

If stock price decreases after buying.

4. What Affects Stock Price?

Stock prices depend on:

- Company profits
- Future expectations

- Number of shares available
- Demand & supply
- Investor sentiment

5. Stock Ticker

A stock ticker continuously shows stock price updates.

Examples:

- TSLA = Tesla
- GME = GameStop
- NFLX = Netflix
- AAPL = Apple

6. Using yfinance Library

What is yfinance?

Python library used to download stock market data from Yahoo Finance.

Used for:

- Historical prices
- Dividends
- Financial metrics

Basic Example

```
import yfinance as yf
msft = yf.Ticker("MSFT")
msft_data = msft.history(period="max")
msft_data.head()
```

Important Period Values

Period Meaning

1d	1 day
5d	5 days
1mo	1 month
6mo	6 months
1y	1 year
5y	5 years
max	Full history

7. Why Web Scraping is Needed

Not all financial data is available through APIs.

Example:

Revenue tables from websites must be scraped manually.

So we use:

- requests
- BeautifulSoup

8. Example Project – Netflix Stock Data

Webpage given contains Netflix historical table.

Need to extract:

- Date
- Open
- High
- Low
- Close
- Volume

Lab: Extracting Stock Data Using a Python Library

1.10 Practice Project: Extracting Stock Data Using Web Scraping

Step 1: Send HTTP Request

```
import requests

url = "website_link"
html_data = requests.get(url).text
```

- `.get()` downloads webpage
- `.text` gives HTML as string

Step 2: Parse HTML

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_data, "html5lib")
```

Parsing Means:

Convert raw HTML into structured tree format.

Step 3: Identify HTML Tags

Web tables use:

Tag	Meaning
<code><table></code>	Table start
<code><tbody></code>	Main content
<code><tr></code>	Row
<code><td></code>	Cell
<code><th></code>	Header

Step 4: Extract Data

Use:

```
find()
find_all()
```

Difference:

- find() → first matching tag
- find_all() → all matching tags

Example:

```
table = soup.find("tbody")
rows = table.find_all("tr")
```

Then loop through rows.

Step 5: Create DataFrame

```
import pandas as pd

df = pd.DataFrame(columns=["Date", "Open", "High", "Low", "Close", "Volume"])
```

Then append extracted rows.

Step 6: View Data

```
df.head()
df.tail()
```

Lab: Extracting Stock Data Using Web Scraping

1.11 Alternative Method – Pandas read_html()

```
tables = pd.read_html(url)
df = tables[0]
```

If webpage has one table, use first table.

1. Short Selling Concept

Short selling means betting that stock price will fall.

Process:

1. Borrow shares
2. Sell at current price
3. Buy back later cheaper
4. Return borrowed shares

Profit:

If price falls.

Loss:

If price rises.

2. Real World Examples

Tesla

Many hedge funds shorted Tesla expecting decline.

Tesla price rose strongly → short sellers lost money.

GameStop Short Squeeze

Retail investors from Reddit **WallStreetBets** bought GameStop heavily.

Price rose sharply.

Hedge funds that shorted GameStop suffered major losses.

3. Your Final Project Task

As Data Scientist:

Extract:

- Tesla stock price
- GameStop stock price
- Tesla revenue
- GameStop revenue

Analyze:

- Price trends
- Revenue trends

Build Dashboard:

Compare:

- Stock price vs Revenue

4. Key Python Tools Used

Library	Purpose
yfinance	Stock price data
requests	Download webpage
BeautifulSoup	Parse HTML
pandas	DataFrames
matplotlib / plotly	Charts

Module 2 – Final Project Notes

2.1 Extracting and Visualizing Stock Data

1. Project Role

You are working as a:

- **Data Scientist**
or
- **Data Analyst**

for a **startup investment firm**.

The firm helps customers make better stock investment decisions.

2. Project Objective

You need to:

Extract Financial Data

From multiple sources using:

- Python libraries
- APIs
- Web scraping

Data Includes:

- Historical stock prices
- Quarterly revenue

Then:

- Visualize data
- Detect patterns
- Build dashboard

3. Stocks Used in Project

You will work on:

- Tesla (**TSLA**)
- Amazon (**AMZN**)
- AMD (**AMD**)
- GameStop (**GME**)

⚠ In assignment questions mainly:

- Tesla
- GameStop

4. Dashboard Analytics

Dashboard helps display important insights clearly.

You will:

- Analyze datasets
- Extract trends
- Create charts
- Present dashboard

5. Graphing Function Given

You are provided function:

```
make_graph(stock_data, revenue_data, stock)
```

Inputs:

Parameter	Meaning
stock_data	DataFrame with Date & Close
revenue_data	DataFrame with Date & Revenue
stock	Stock Name

6. What Graph Shows

Two graphs:

Graph 1:

Historical Share Price

Graph 2:

Historical Revenue

2.2 Question Wise Notes

Question 1: Tesla Stock Data using yfinance

Task:

Use Tesla ticker:

TSLA

Code:

```
import yfinance as yf
tesla = yf.Ticker("TSLA")
tesla_data = tesla.history(period="max")
tesla_data.reset_index(inplace=True)
tesla_data.head()
```

Output:

First 5 rows.

Question 2: Tesla Revenue using Web Scrapping

URL:

Revenue webpage:

<https://cf-courses-data.s3.../revenue.htm>

Steps:

1. Download page

```
import requests
html_data = requests.get(url).text
```

2. Parse HTML

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_data, "html.parser")
```

3. Extract Table

Use:

```
pd.read_html(html_data)

Tesla revenue table is:

tables[1]
```

Cleaning Revenue Column

```
tesla_revenue.columns = ["Date", "Revenue"]
tesla_revenue["Revenue"] = tesla_revenue["Revenue"].str.replace(",|\$", "", regex=True)
tesla_revenue.dropna(inplace=True)
tesla_revenue = tesla_revenue[tesla_revenue["Revenue"] != ""]
```

Show Output

```
tesla_revenue.tail()
```

Question 3: GameStop Stock Data

Ticker:

GME

Code:

```
gme = yf.Ticker("GME")
gme_data = gme.history(period="max")
gme_data.reset_index(inplace=True)
gme_data.head()
```

Question 4: GameStop Revenue Data

URL:

```
stock.html
```

Code:

```
html_data_2 = requests.get(url).text
tables = pd.read_html(html_data_2)
gme_revenue = tables[1]
```

Clean Data

```
gme_revenue.columns = ["Date","Revenue"]  
gme_revenue["Revenue"] = gme_revenue["Revenue"].str.replace(",|\$", "", regex=True)  
gme_revenue.dropna(inplace=True)  
gme_revenue = gme_revenue[gme_revenue["Revenue"] != ""]
```

Show Output

```
gme_revenue.tail()
```

Question 5: Tesla Graph

Use:

```
make_graph(tesla_data, tesla_revenue, "Tesla")
```

Shows:

- Tesla Share Price
- Tesla Revenue

(up to June 2021)

Question 6: GameStop Graph

Use:

```
make_graph(gme_data, gme_revenue, "GameStop")
```

Shows:

- GME Share Price
- GME Revenue

2.3 Most Important Commands

Reset Index

```
reset_index(inplace=True)
```

Converts Date index into normal column.

Read HTML Tables

```
pd.read_html(html_data)
```

Remove Symbols

```
str.replace(",|\\$", "", regex=True)
```

Removes:

- commas
- dollar signs

Remove Empty Rows

```
dropna()
```

Expected Trend Insight

Tesla:

- Stock price surged strongly by 2021
- Revenue increased steadily

GameStop:

- Stock price spiked sharply in 2021
- Revenue did not match same surge

Analyzing Historical Stock/Revenue Data and Building a Dashboard
